

Periféricos y Dispositivos de Interfaz Humana

Universidad de Granada - Curso 2025/2026

Práctica 3: Experimentación con Arduino

Por Alejandro López Jiménez

Índice:

- 1. Introducción
- 2. Requisitos mínimos
 - 2.1 Ejercicio 1
 - 2.2 Ejercicio 2
- 3. Requisitos ampliados
 - 3.1 Ejercicio 3
 - 3.2 Ejercicio 4

-1. Introducción

El objetivo de esta práctica es obtener nociones básicas en el funcionamiento de Arduino, creando circuitos primero en el simulador Tinkercad para verificar su correcto funcionamiento y luego implementarlos en un circuito real cuya lógica se define en el Arduino IDE, un software que le indica a la placa como debe comportarse.

-2. Requisitos mínimos

En los requisitos mínimos de esta práctica se proponen dos ejercicios básicos.

El primero consiste en hacer parpadear alternativamente 3 LEDs de diferentes colores (rojo, amarillo y verde) para que se enciendan y apaguen alternativamente simulando las luces de un semáforo. Dichos LEDs irán conectados a las salidas digitales 11, 12 y 13 y el intervalo de tiempo con el que alternan su activación será de 1,5 segundos.

El segundo ejercicio parte del anterior y consiste en añadir un pulsador conectado a la salida 7 de modo que el LED rojo solo se encienda cuando el pulsador se activa y al mismo tiempo los LEDs amarillo y verde se apaguen.

-2.1 Ejercicio 1:

En el primer he asignado los pines 13, 12 y 11 a los LEDs rojo, amarillo y verde respectivamente, que a su vez estan conectados cada uno a tierra y a una resistencia, mediante un código sencillo he hecho que se enciendan en secuencia con un retardo de 1,5s.

Enlace a la implementación en Tinkercad:

<https://www.tinkercad.com/things/98MJA04m5lP-ejercicio-1-practica-3-pdih>

Código:

```
1. void setup() {
2.   //Configuración inicial:
3.   pinMode(13, OUTPUT); //pin 13 conectado al LED rojo
4.   pinMode(12, OUTPUT); //pin 12 conectado al LED amarillo
5.   pinMode(11, OUTPUT); //pin 11 conectado al LED verde
6. }
7.
8. void loop() {
9.   //Ejecución en bucle:
10.  digitalWrite(13, HIGH); //encendemos el LED rojo
11.  digitalWrite(12, LOW); //apagamos el LED amarillo
12.  digitalWrite(11, LOW); //apagamos el LED verde
13.  delay(1500); //esperamos 1500 ms (1,5 segundos)
14.  digitalWrite(13, LOW); //apagamos el LED rojo
15.  digitalWrite(12, HIGH); //encendemos el LED amarillo
16.  digitalWrite(11, LOW); //apagamos el LED verde
17.  delay(1500); //esperamos 1500 ms
18.  digitalWrite(13, LOW); //apagamos el LED rojo
19.  digitalWrite(12, LOW); //apagamos el LED amarillo
20.  digitalWrite(11, HIGH); //encendemos el LED verde
21.  delay(1500); //esperamos 1500 ms
22. }
```

Captura de Tinkercad:

TINKERCAD Ejercicio 1 - Practica 3 PDIH Se han guardado todos los cambios. Hora de simulador: 00:00:09

1 (Arduino Uno R3)

```
1 void setup() {
2   //Configuración inicial:
3   pinMode(13, OUTPUT); //pin 13 conectado al LED rojo
4   pinMode(12, OUTPUT); //pin 12 conectado al LED amarillo
5   pinMode(11, OUTPUT); //pin 11 conectado al LED verde
6 }
7
8 void loop() {
9   //Ejecución en bucle:
10  digitalWrite(13, HIGH); //encendemos el LED rojo
11  digitalWrite(12, LOW); //apagamos el LED amarillo
12  digitalWrite(11, LOW); //apagamos el LED verde
13  delay(1500); //esperamos 1500 ms (1,5 segundos)
14  digitalWrite(13, LOW); //apagamos el LED rojo
15  digitalWrite(12, HIGH); //encendemos el LED amarillo
16  digitalWrite(11, LOW); //apagamos el LED verde
17  delay(1500); //esperamos 1500 ms
18  digitalWrite(13, LOW); //apagamos el LED rojo
19  digitalWrite(12, LOW); //apagamos el LED amarillo
20  digitalWrite(11, HIGH); //encendemos el LED verde
21  delay(1500); //esperamos 1500 ms
22 }
23
```

Activar Windows
Ve a Configuración para activar Windows.

Monitor en serie

Esquema:

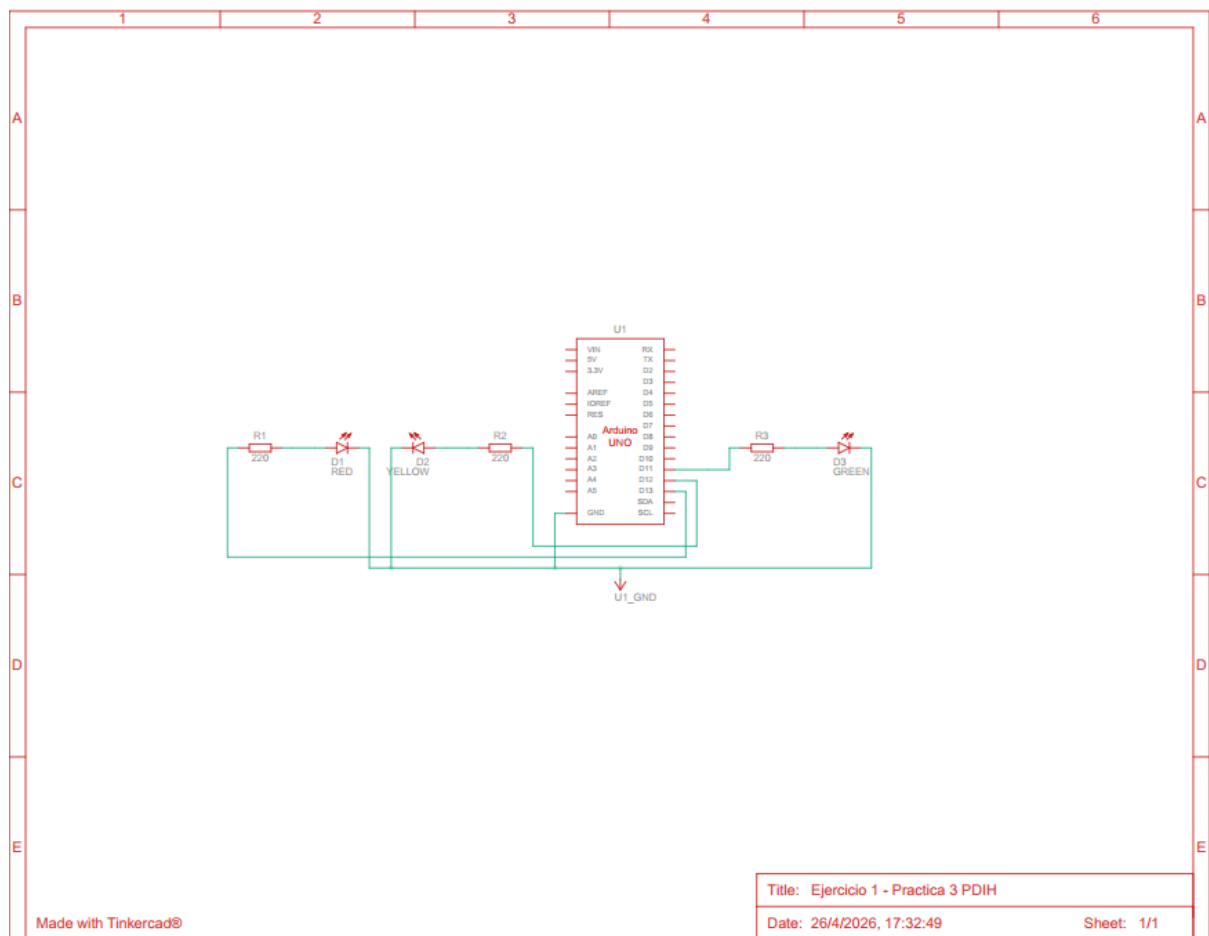
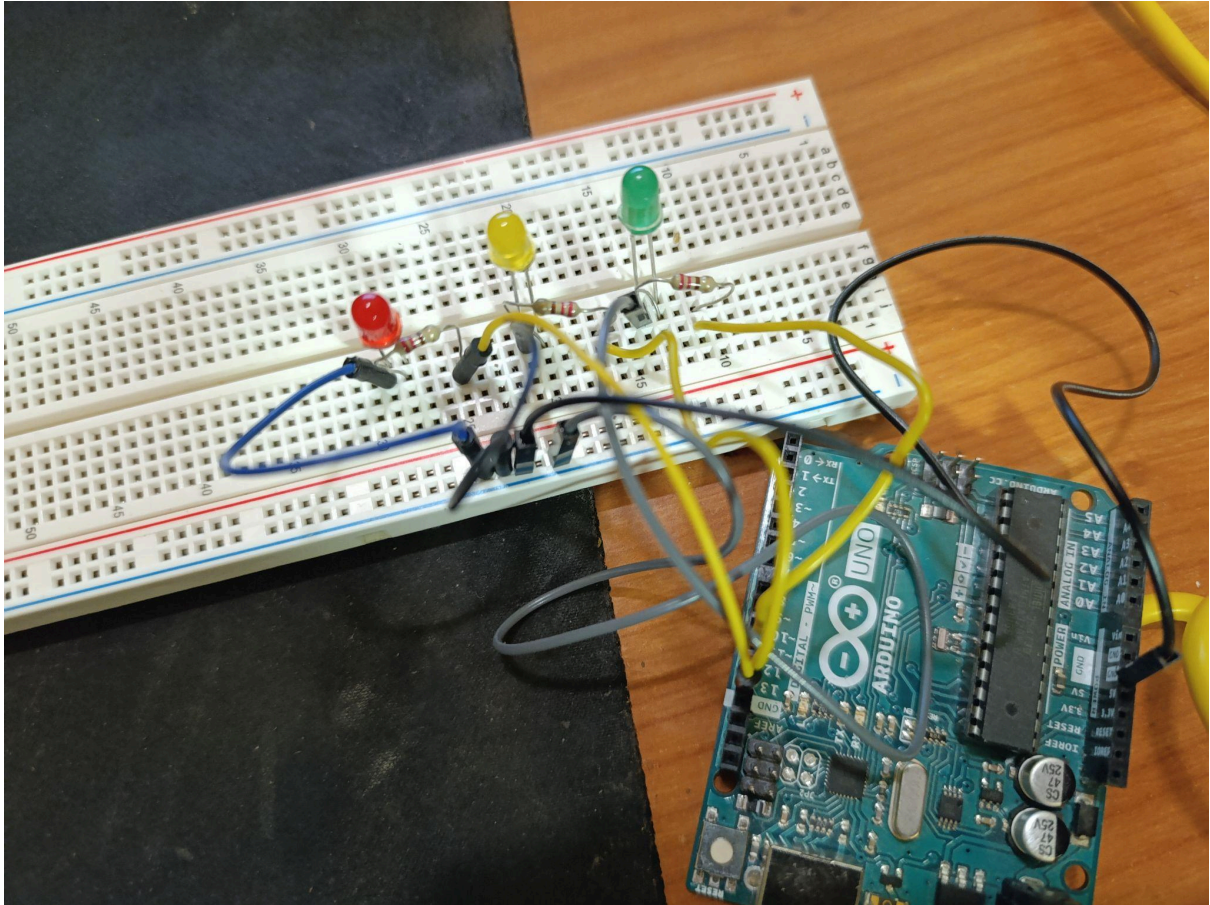


Foto del circuito:



En la carpeta de videos hay dos vídeos mostrando cómo funciona en una primera y segunda implementación del circuito.

-2.2 Ejercicio 2:

Para el segundo ejercicio he partido del diseño anterior simplemente añadiendo un pulsador conectado al pin 7 y a su vez a tierra y a una resistencia. Al código anterior le he añadido una condición para comprobar si el pulsador está presionado y en dicho caso se encienda el LED rojo y se apaguen los demás.

Enlace a la implementación en Tinkercad:

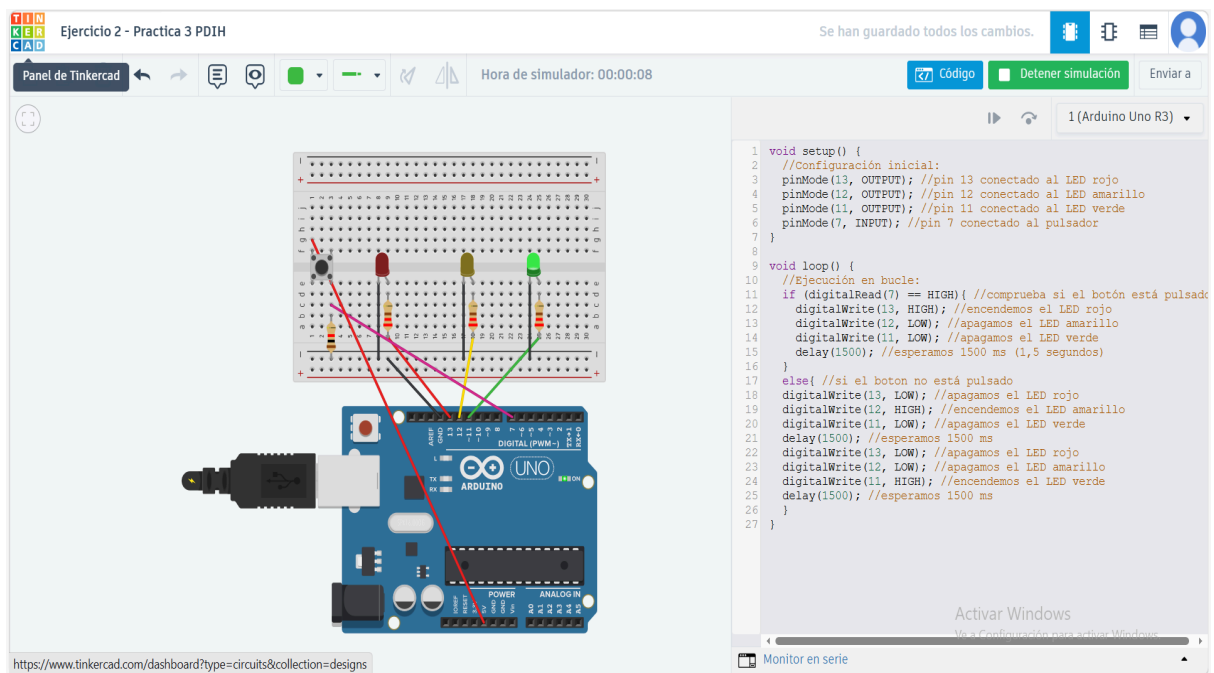
https://www.tinkercad.com/things/eFVdrQi9tae-ejercicio-2-practica-3-pdi_h

Código:

```
1. void setup() {
2.   //Configuración inicial:
3.   pinMode(13, OUTPUT); //pin 13 conectado al LED rojo
4.   pinMode(12, OUTPUT); //pin 12 conectado al LED amarillo
5.   pinMode(11, OUTPUT); //pin 11 conectado al LED verde
6.   pinMode(7, INPUT); //pin 7 conectado al pulsador
7. }
8.
9. void loop() {
10.  //Ejecución en bucle:
11.  if (digitalRead(7) == HIGH){ //comprueba si el botón está pulsado
12.    digitalWrite(13, HIGH); //encendemos el LED rojo
13.    digitalWrite(12, LOW); //apagamos el LED amarillo
14.    digitalWrite(11, LOW); //apagamos el LED verde
15.    delay(1500); //esperamos 1500 ms (1,5 segundos)
16.  }
17.  else{ //si el boton no está pulsado
18.    digitalWrite(13, LOW); //apagamos el LED rojo
19.    digitalWrite(12, HIGH); //encendemos el LED amarillo
20.    digitalWrite(11, LOW); //apagamos el LED verde
```

```
21. delay(1500); //esperamos 1500 ms
22. digitalWrite(13, LOW); //apagamos el LED rojo
23. digitalWrite(12, LOW); //apagamos el LED amarillo
24. digitalWrite(11, HIGH); //encendemos el LED verde
25. delay(1500); //esperamos 1500 ms
26. }
27. }
28.
```

Captura de Tinkercad:



Esquema:

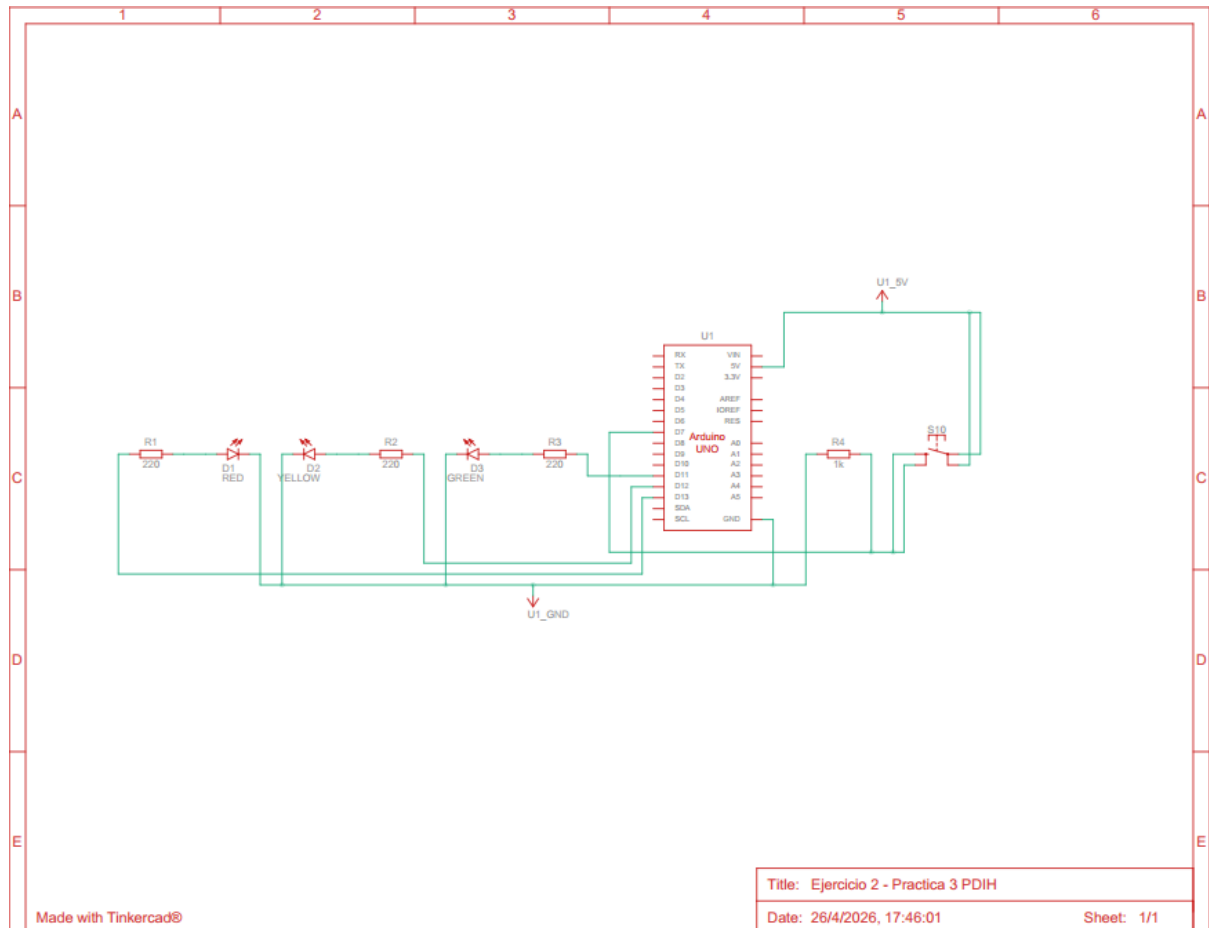
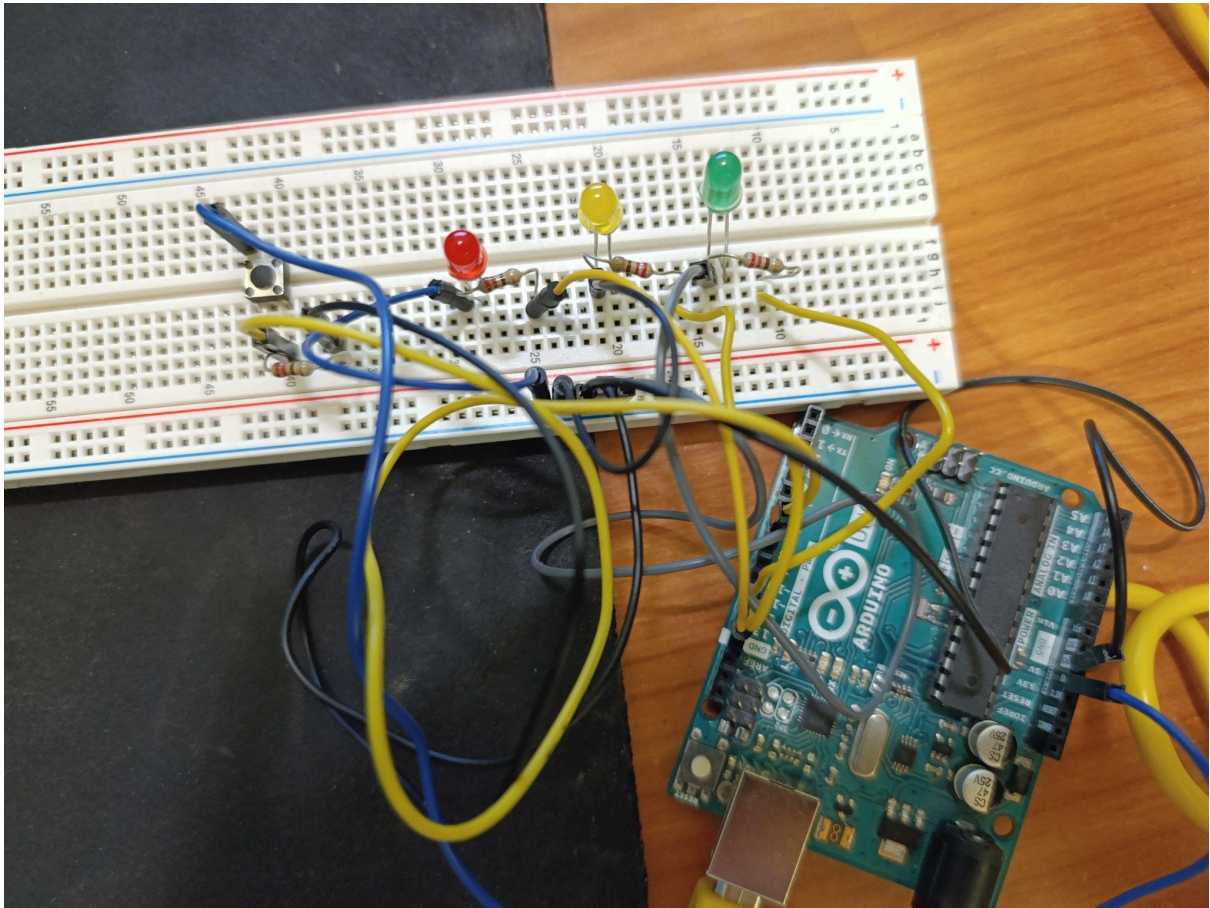


Foto del circuito:



En la carpeta de videos hay un vídeo mostrando cómo funciona.

-3. Requisitos ampliados

Para los requisitos ampliados se proponen varios ejercicios, de los cuales he realizado los dos primeros.

El primero (Ejercicio 3) consiste en hacer una secuencia luz usando 4 LEDs rojos que imite las luces del coche de la mítica serie “El coche fantástico”.

El segundo trata de usar el detector de movimiento por ultrasonidos HC-SR04 para detectar un objeto y en función de la distancia a la que se encuentre ese objeto hacer que un altavoz emita un pitido en diferentes frecuencias.

-3.1 Ejercicio 3

Para hacer este ejercicio he usado 4 LEDs rojos conectados a los pines 13, 12, 11 y 10, nuevamente conectados a su vez a tierra y a 4 resistencias.

Para crear el efecto he hecho un bucle envolvente en el que se van encendiendo en secuencia, con un retardo de 0,2 segundos, y al encenderse el último se sigue el orden inversamente hasta acabar de nuevo en el primero.

Enlace a la implementación en Tinkercad:

<https://www.tinkercad.com/things/gM9mvqLUZ8D-ejercicio-3-practica-3-pdih>

Código:

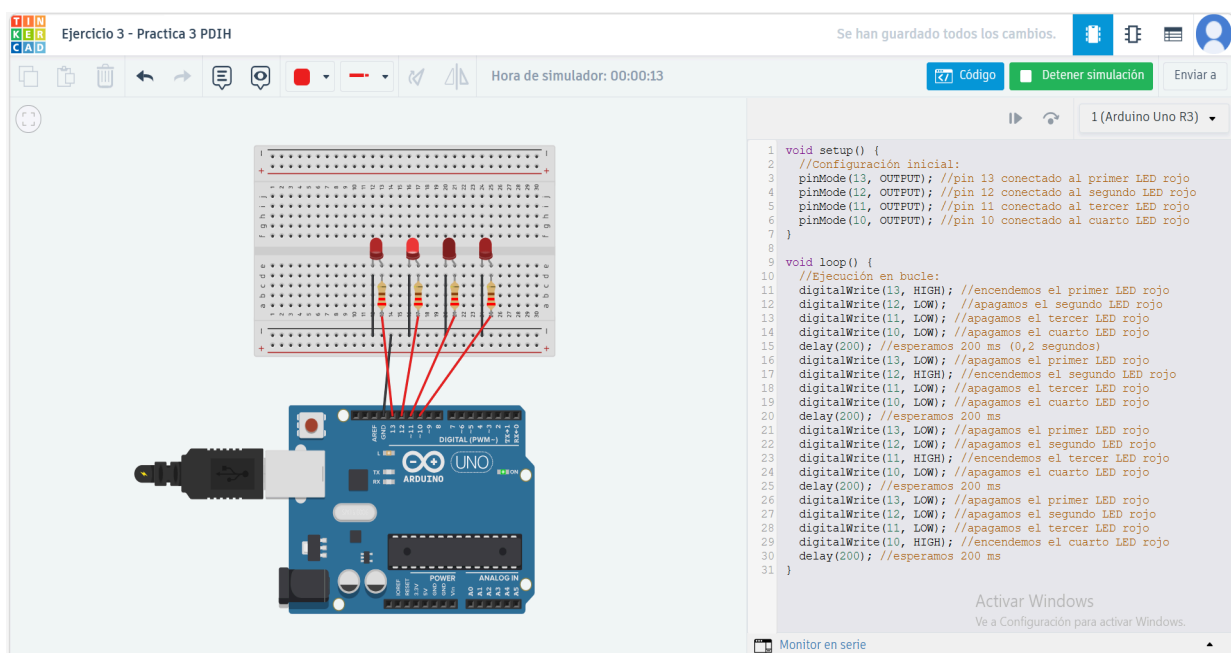
```
1. void setup() {
2.   //Configuración inicial:
3.   pinMode(13, OUTPUT); //pin 13 conectado al primer LED rojo
4.   pinMode(12, OUTPUT); //pin 12 conectado al segundo LED rojo
5.   pinMode(11, OUTPUT); //pin 11 conectado al tercer LED rojo
6.   pinMode(10, OUTPUT); //pin 10 conectado al cuarto LED rojo
7. }
8.
9. void loop() {
10.  //Ejecución en bucle:
11.  digitalWrite(13, HIGH); //encendemos el primer LED rojo
12.  digitalWrite(12, LOW); //apagamos el segundo LED rojo
13.  digitalWrite(11, LOW); //apagamos el tercer LED rojo
14.  digitalWrite(10, LOW); //apagamos el cuarto LED rojo
15.  delay(200); //esperamos 200 ms (0,2 segundos)
16.  digitalWrite(13, LOW); //apagamos el primer LED rojo
17.  digitalWrite(12, HIGH); //encendemos el segundo LED rojo
18.  digitalWrite(11, LOW); //apagamos el tercer LED rojo
```

```

19. digitalWrite(10, LOW); //apagamos el cuarto LED rojo
20. delay(200); //esperamos 200 ms
21. digitalWrite(13, LOW); //apagamos el primer LED rojo
22. digitalWrite(12, LOW); //apagamos el segundo LED rojo
23. digitalWrite(11, HIGH); //encendemos el tercer LED rojo
24. digitalWrite(10, LOW); //apagamos el cuarto LED rojo
25. delay(200); //esperamos 200 ms
26. digitalWrite(13, LOW); //apagamos el primer LED rojo
27. digitalWrite(12, LOW); //apagamos el segundo LED rojo
28. digitalWrite(11, LOW); //apagamos el tercer LED rojo
29. digitalWrite(10, HIGH); //encendemos el cuarto LED rojo
30. delay(200); //esperamos 200 ms
31. digitalWrite(13, LOW); //apagamos el primer LED rojo
32. digitalWrite(12, LOW); //apagamos el segundo LED rojo
33. digitalWrite(11, HIGH); //encendemos el tercer LED rojo
34. digitalWrite(10, LOW); //apagamos el cuarto LED rojo
35. delay(200); //esperamos 200 ms
36. digitalWrite(13, LOW); //apagamos el primer LED rojo
37. digitalWrite(12, HIGH); //encendemos el segundo LED rojo
38. digitalWrite(11, LOW); //apagamos el tercer LED rojo
39. digitalWrite(10, LOW); //apagamos el cuarto LED rojo
40. delay(200); //esperamos 200 ms
41. }

```

Captura de Tinkercad:



Esquema:

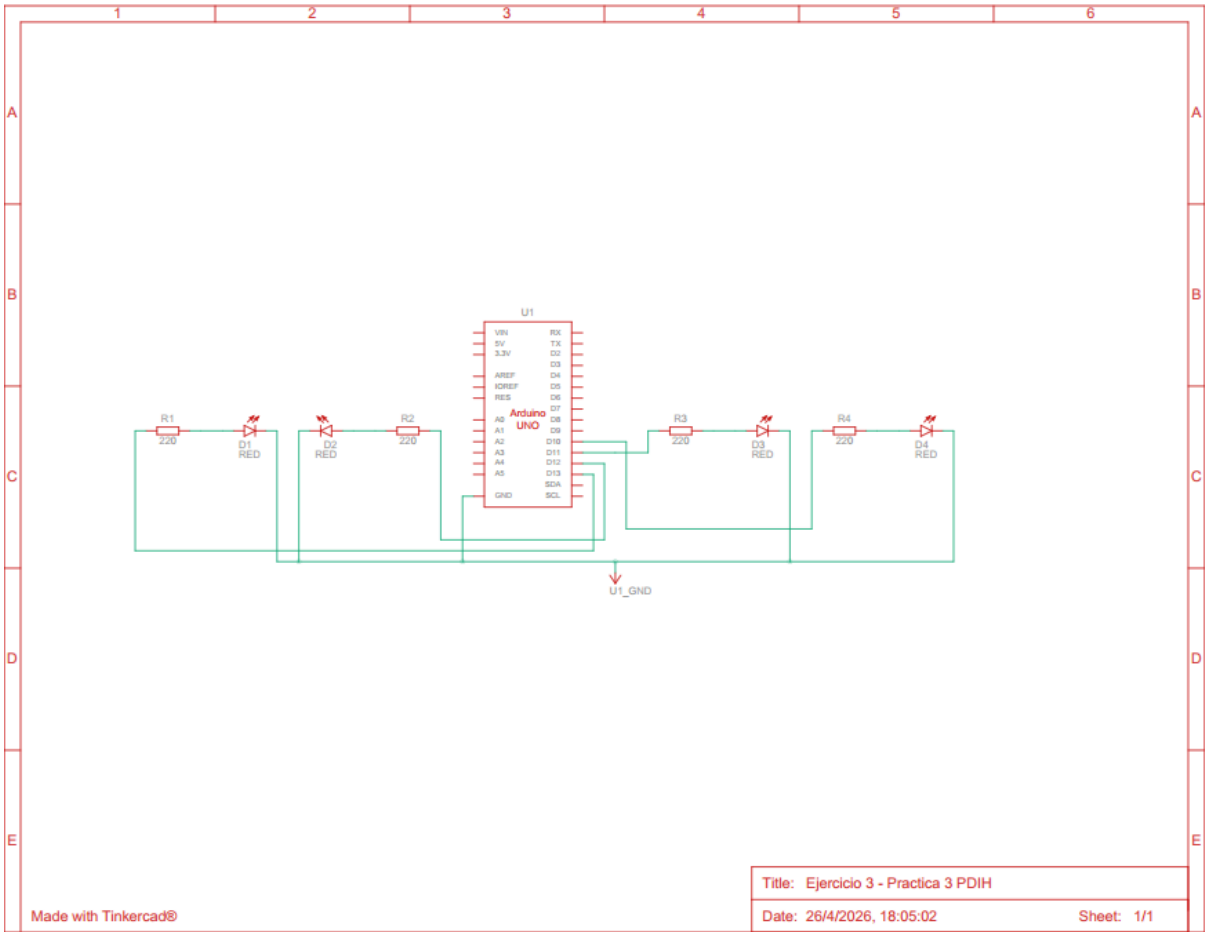
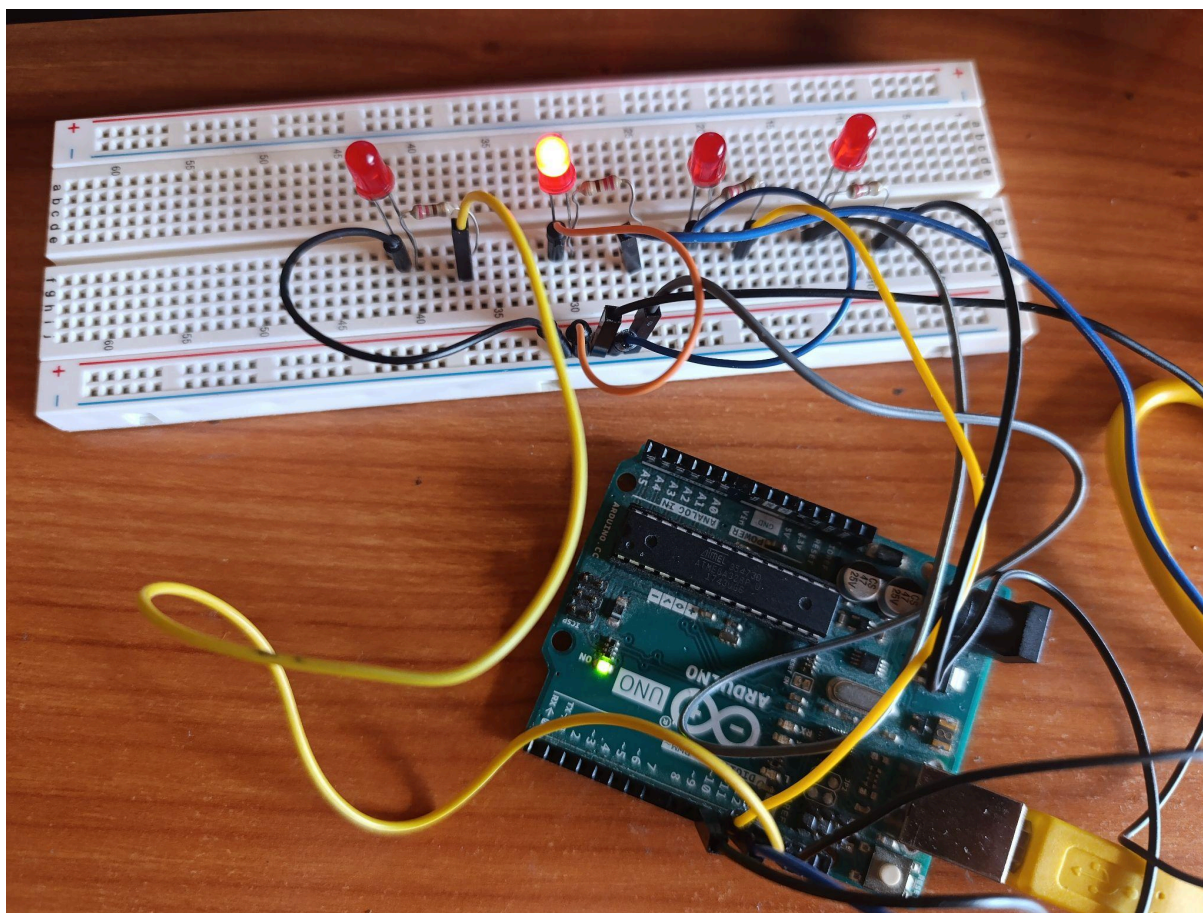


Foto del circuito:



En la carpeta de videos hay un vídeo mostrando cómo funciona.

-3.2 Ejercicio 4

El sensor HC-SR04 utilizado en este ejercicio cuenta con entradas para el disparador (TRIG), el cual emite la onda que rebota, y el receptor (ECHO), que la recibe para medir cuánto ha tardado y poder así medir la distancia. El TRIG lo he conectado al pin 12 y el ECHO al 13. Lo he conectado a tierra y le he asignado un voltaje de 5V.

Por otro lado, el altavoz que emite el pitido lo he conectado al pin 11 y a tierra. No ha sido necesario usar resistencias en este diseño.

Mediante una simple fórmula matemática ($\text{Duración del rebote} * 0.034 / 2$) obtengo la distancia a la que se encuentra el objeto y la almaceno en una variable para poder imprimirla por pantalla y condicionar la frecuencia a la que el altavoz pita mediante simples “ifs” y “elses”.

Enlace a la implementación en Tinkercad:

<https://www.tinkercad.com/things/eN1P4lu66mZ-ejercicio-4-practica-3-pdih>

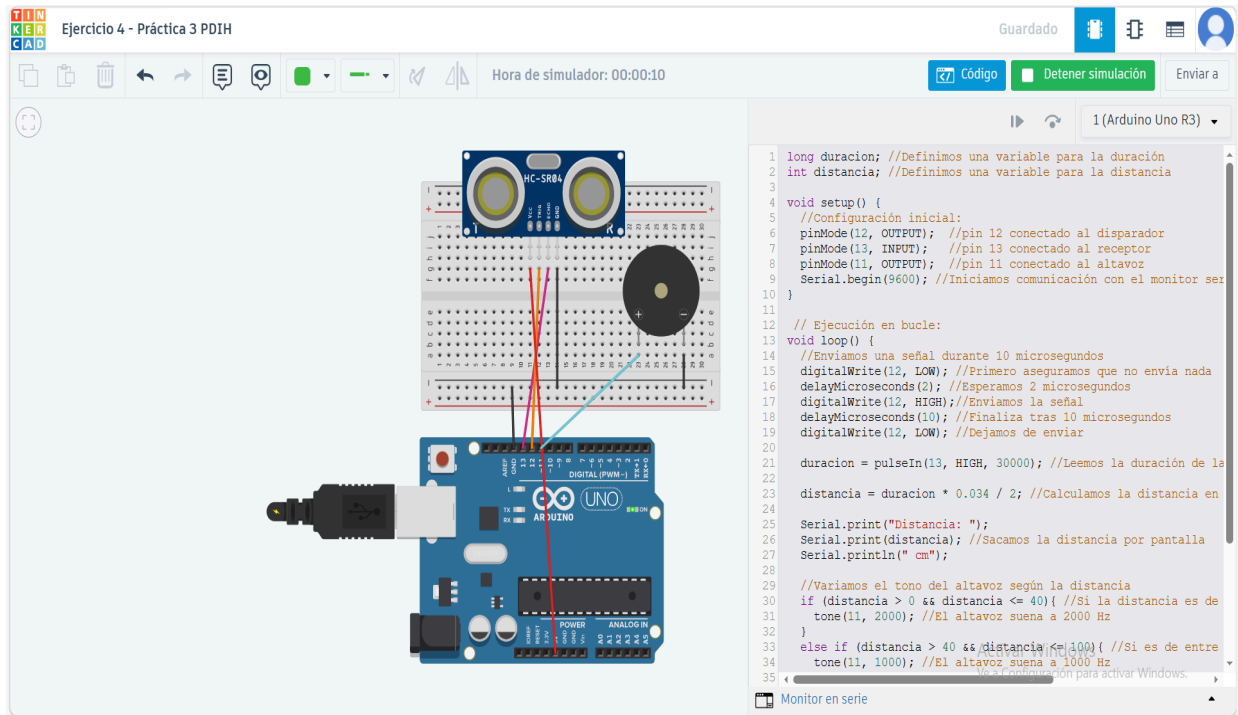
Código:

```
1. long duracion; //Definimos una variable para la duración
2. int distancia; //Definimos una variable para la distancia
3.
4. void setup() {
5.   //Configuración inicial:
6.   pinMode(12, OUTPUT); //pin 12 conectado al disparador
7.   pinMode(13, INPUT); //pin 13 conectado al receptor
8.   pinMode(11, OUTPUT); //pin 11 conectado al altavoz
9.   Serial.begin(9600); //Iniciamos comunicación con el monitor serie (para poder
    imprimir las distancias)
10. }
11.
```



```
12. // Ejecución en bucle:
13. void loop() {
14. //Enviamos una señal durante 10 microsegundos
15. digitalWrite(12, LOW); //Primero aseguramos que no envía nada
16. delayMicroseconds(2); //Esperamos 2 microsegundos
17. digitalWrite(12, HIGH); //Enviamos la señal
18. delayMicroseconds(10); //Finaliza tras 10 microsegundos
19. digitalWrite(12, LOW); //Dejamos de enviar
20.
21. duracion = pulseIn(13, HIGH, 30000); //Leemos la duración de la señal
22.
23. distancia = duracion * 0.034 / 2; //Calculamos la distancia en cm
24.
25. Serial.print("Distancia: ");
26. Serial.print(distancia); //Sacamos la distancia por pantalla
27. Serial.println(" cm");
28.
29. //Variamos el tono del altavoz según la distancia
30. if (distancia > 0 && distancia <= 40){ //Si la distancia es de entre 0 y 40 cm
31.   tone(11, 2000); //El altavoz suena a 2000 Hz
32. }
33. else if (distancia > 40 && distancia <= 100){ //Si es de entre 40 y 100 cm
34.   tone(11, 1000); //El altavoz suena a 1000 Hz
35. }
36. else if (distancia > 100 && distancia <= 150){ //Entre 100 y 150 cm
37.   tone(11, 500); //El altavoz suena a 500 Hz
38. }
39. else{
40.   noTone(11); // En cualquier otro caso no suena
41. }
42.
43. delay(150); //Esperamos antes de que el sensor dispare de nuevo
44. }
45.
```

Captura de Tinkercad:



Esquema:

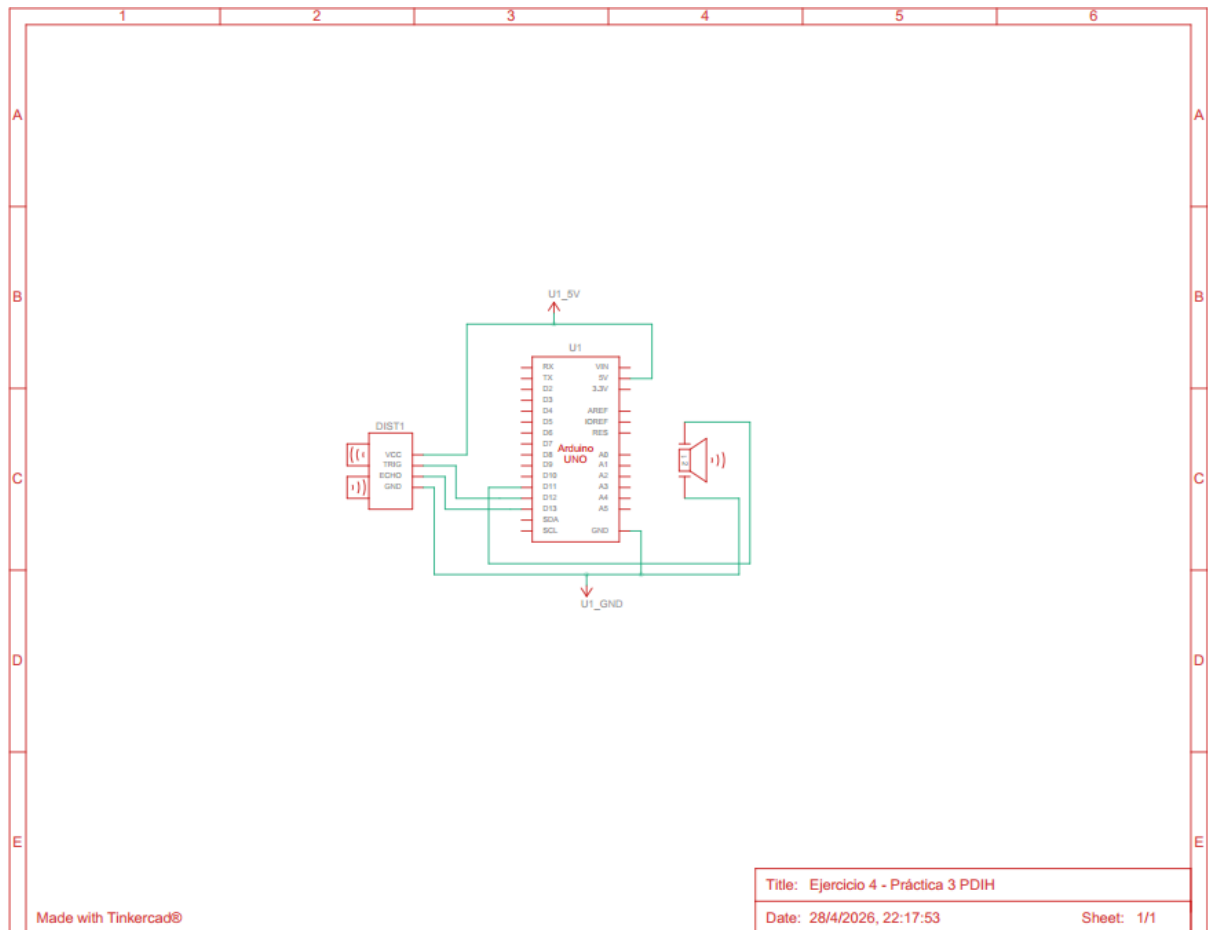
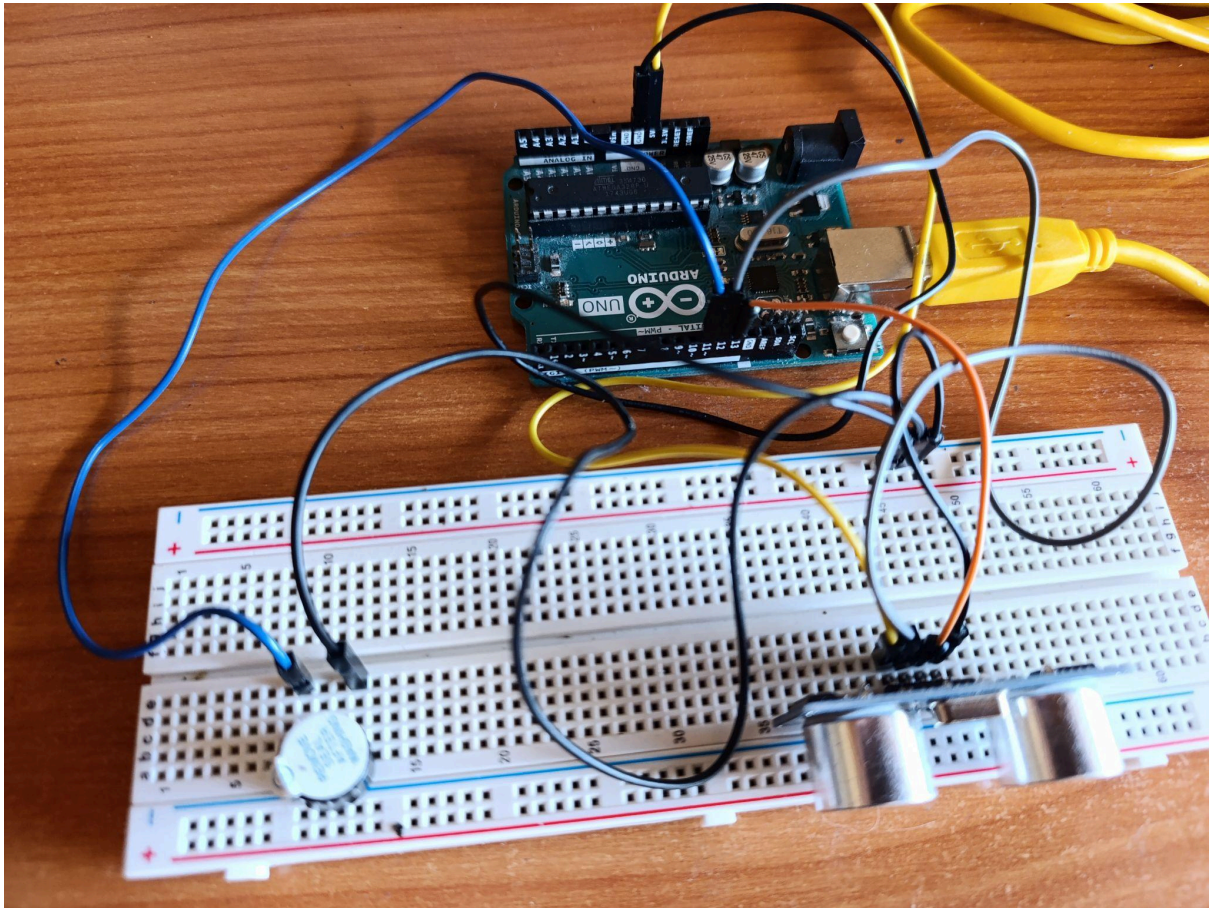


Foto del circuito:



En la carpeta de videos hay un vídeo mostrando cómo funciona.

Fin